LDSCI7229 Advanced Data Engineering - AE1 Report

GitHub: https://github.com/AdvDataEng123/advanced-data-engineering-ae1

> **Note on the screenshots.** A few of the AWS console screenshots in this report have small black boxes over my name and account details. I only realised after I'd already finished the assignment that some of the captures still showed my AWS Academy user label, my 12-digit Account ID, and the bucket suffix that included my GitHub handle, so I went back and blacked out just those bits for anonymity. Everything else in the screenshots (config values, query results, execution status) is untouched, and none of the redacted values appear anywhere in the code or configs in this repo.

Task 1 - Serverless Ingestion Pipeline

1. Dataset explanation

For Task 1 I designed a serverless ingestion pipeline around two deliberately contrasting datasets so I could exercise both batch and streaming patterns inside one architecture.

The first is the **US DOT Border Crossing Entry Data**, published by the US Bureau of Transportation Statistics. It contains roughly 273,000 rows dating back to 1996, where each row represents the monthly count of vehicles, pedestrians, or passengers passing through a US land port. Columns include port name, state, border (US-Canada or US-Mexico), date, transport measure, value, and a geographic point. It is a static historical dataset that updates infrequently, making it a natural fit for batch ingestion.

The second is **OpenAlex**, a free open academic graph exposing a REST API for research papers, authors, institutions, and citations. Each "work" is returned as a nested JSON object containing arrays of authors, institutions, and per-year citation counts. Because OpenAlex is continuously updated and best consumed via API calls rather than bulk dumps, I treated it as a streaming source.

Basically these two sources are different in every way: CSV vs JSON, flat vs nested, one-shot download vs continuous API calls. That contrast is why I picked them, since supporting both patterns in one pipeline forces the architecture to be more flexible.

2. Pipeline design principles

The pipeline is fully serverless, a choice driven both by the AWS Academy Learner Lab constraints (no custom IAM, no VPCs, no Bedrock/Redshift/EMR) and by the broader goal of building something that scales without provisioning compute by hand.

I structured the S3 data lake into three zones: `raw/` for untouched source data, `processed/` reserved for staging, and `cleaned/` for validated Parquet output (**Figure 1**). This separation guarantees that raw data is never overwritten, so I can reprocess at any time without re-ingesting from the source.

For streaming, a Lambda function (`openalex-ingestion`) calls the OpenAlex API in pages and forwards each page as a JSON record to a Kinesis Data Firehose stream (**Figure 2**). Firehose buffers up to five minutes or 5 MB of records before writing a batch file to `raw/openalex/`. The Lambda is triggered hourly by an EventBridge schedule, so the lake fills incrementally without me touching anything.

For batch, the border crossing CSV is uploaded directly to `raw/`. Two AWS Glue jobs (`border-crossing-etl` and `openalex-etl`) then read each dataset, clean it, and write Parquet output to `cleaned/`.

The entire flow is wrapped in a single Step Functions state machine called `data-pipeline` (**Figure 3**). It first invokes the Lambda, then runs both Glue jobs in parallel using a `Parallel` state, and finally records success in DynamoDB. I chose **Standard** Step Functions rather than Express, because Express workflows have a five-minute hard execution limit and my Glue jobs routinely run longer than that - Standard also gives me a full execution history I can inspect retroactively, which proved useful when debugging failed runs. Every step has `Retry` for transient errors and `Catch` blocks that route to a `PipelineFailed` terminal state, so the error handling is built into the orchestration layer rather than requiring additional code. The result is a pipeline I can launch end-to-end with one click and that gracefully recovers from most transient failures.

3. Streaming vs batch ingestion trade-offs

Batch and streaming suit different use cases. The key trade-off is data freshness versus operational complexity.

Batch ingestion (border crossing CSV) is simple and predictable. There is no buffering, no event-driven plumbing, and no race conditions: a file lands in S3 and Glue picks it up. The trade-off is freshness, since the data is only as current as the last upload, which makes batch unsuitable for anything requiring real-time insight. Reprocessing is trivial because the input is a single file; rerunning the Glue job with `mode("overwrite")` won’t create duplicates if you run it twice. Cost is low because S3 storage is cheap and the Glue job only runs on demand.

Streaming ingestion (OpenAlex via Lambda and Firehose) gives much fresher data but introduces several new failure modes. Lambda timeouts, Firehose throttling, EventBridge scheduling drift, and OpenAlex rate limits all become things I need to consider. Debugging is harder because instead of one well-defined input file there are hundreds of small JSON objects arriving asynchronously. I deliberately used Firehose rather than having Lambda write directly to S3 because Firehose batches records into larger files every five minutes, which cuts down on the small-file problem that would slow Glue down later.

The border crossing dataset is static and analytical, so the simplicity of batch wins. OpenAlex is continuously updated and the use case (citation trends, recent papers) benefits from freshness, so the extra complexity of streaming is justified. Having both in one pipeline added complexity, but

it forced me to make the downstream cleaning and metadata logging generic enough to work for either pattern.

4. Data transformation and quality control considerations

Both Glue ETL jobs use PySpark and follow the same shape: extract from `raw/`, transform the schema, validate row by row, and write to `cleaned/` as Parquet.

The `border-crossing-etl` script reads the CSV with header inference, drops a redundant column, parses the `Date` field into a proper timestamp, casts numeric counts to integers, and drops any rows with null values in key columns (port, value, date). Malformed rows (the source CSV does contain a few) are silently discarded rather than allowed to poison downstream queries.

The `openalex-etl` script is more involved because the source is deeply nested JSON. It reads every JSON file under `raw/openalex/`, then uses Spark's `select`, `explode`, and `getField` operations to flatten the structure into a flat schema (title, publication date, citation count, primary author, primary institution, country). Missing fields coalesce to nulls rather than failing the job.

Both scripts write to `cleaned/` as **Parquet with Snappy compression**. Parquet is columnar, so the Athena queries in Task 2 only scan the columns they need, which makes the warehouse layer cheap and fast. Using `mode("overwrite")` keeps every job idempotent, so reruns produce the same result rather than appending duplicates.

Quality control extends beyond the scripts themselves. Each Glue job writes a record to a DynamoDB table called `pipeline-metadata` after each run, capturing job name, run timestamp, source path, rows processed, rows cleaned, duration, and status (**Figure 4**). This gives me a log I can always go back to. I can answer "did this job run, how many rows, how long did it take" without digging through CloudWatch.

Finally, the Step Functions `Catch` blocks catch anything that goes wrong. If a step throws an error the whole pipeline stops and you can see exactly where it failed in the graph (**Figure 5**), so failures are caught at the orchestration layer rather than silently producing bad data.

Figures

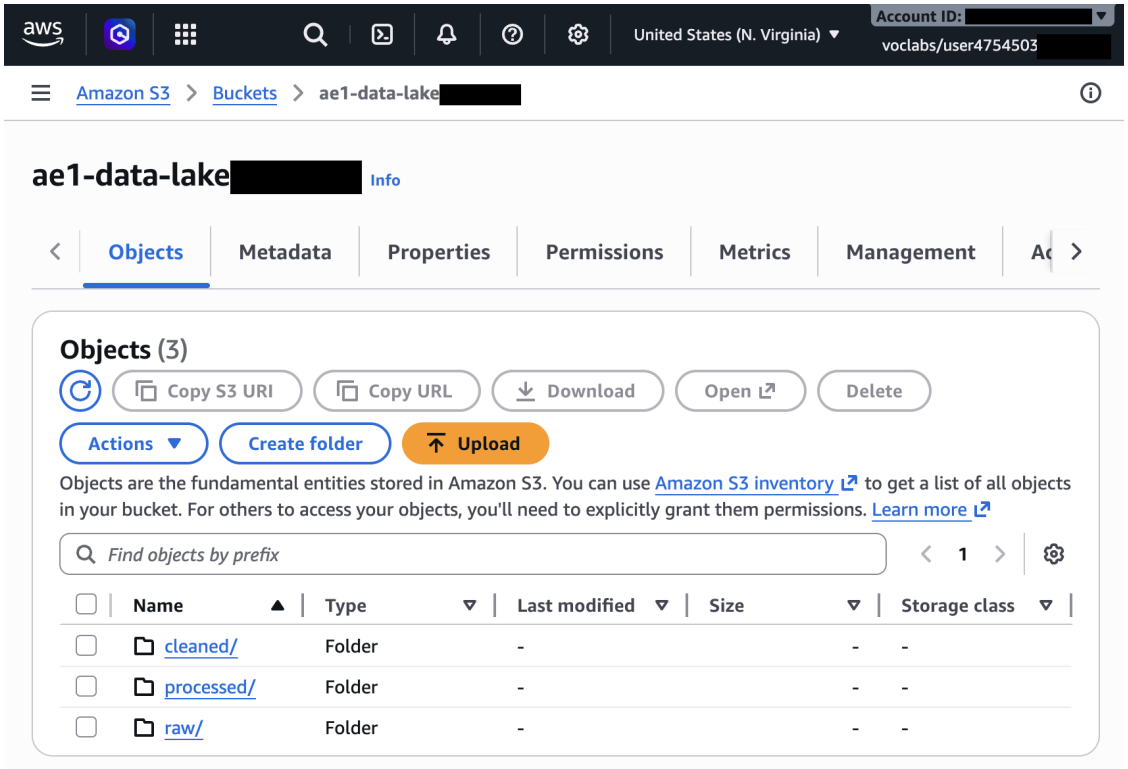


Figure 1: S3 data lake showing raw/, processed/, cleaned/ zones

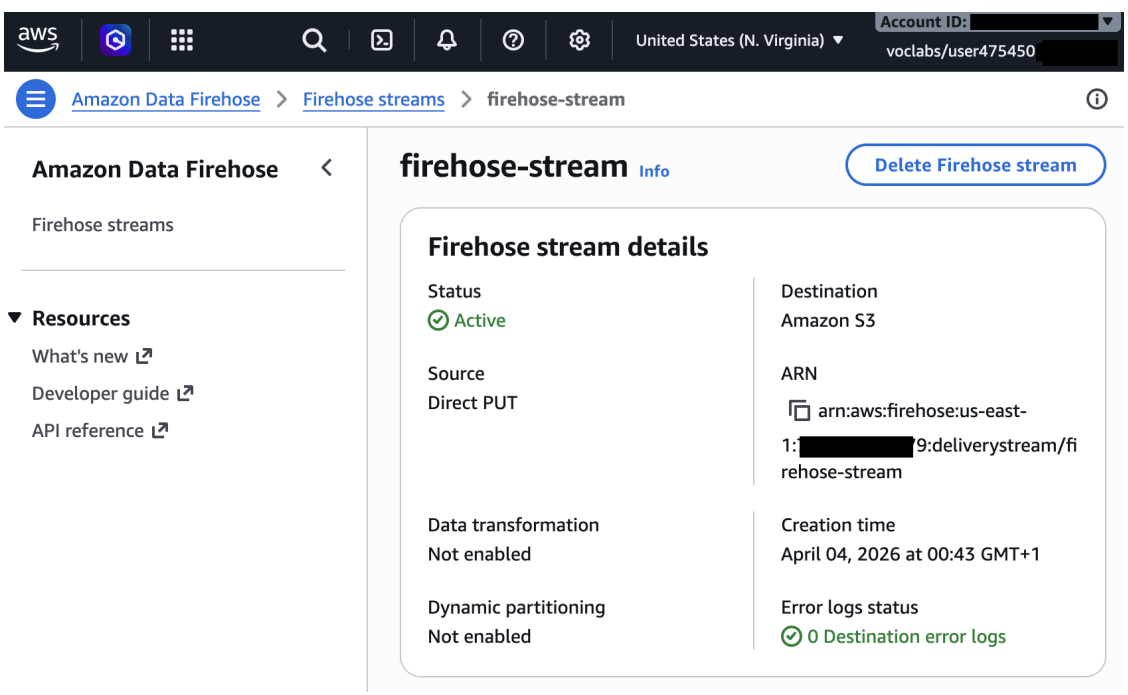


Figure 2: Kinesis Data Firehose delivery stream configured to write to raw/openalex/

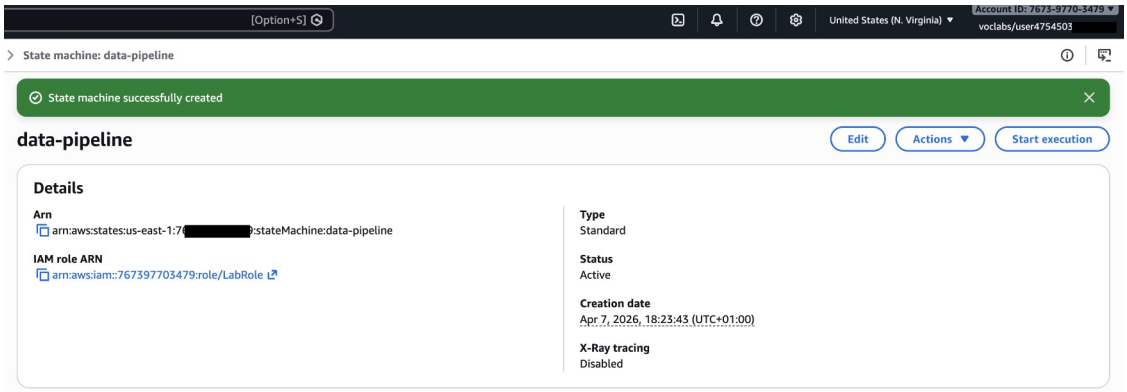


Figure 3: data-pipeline Step Functions state machine, Type: Standard, Status: Active

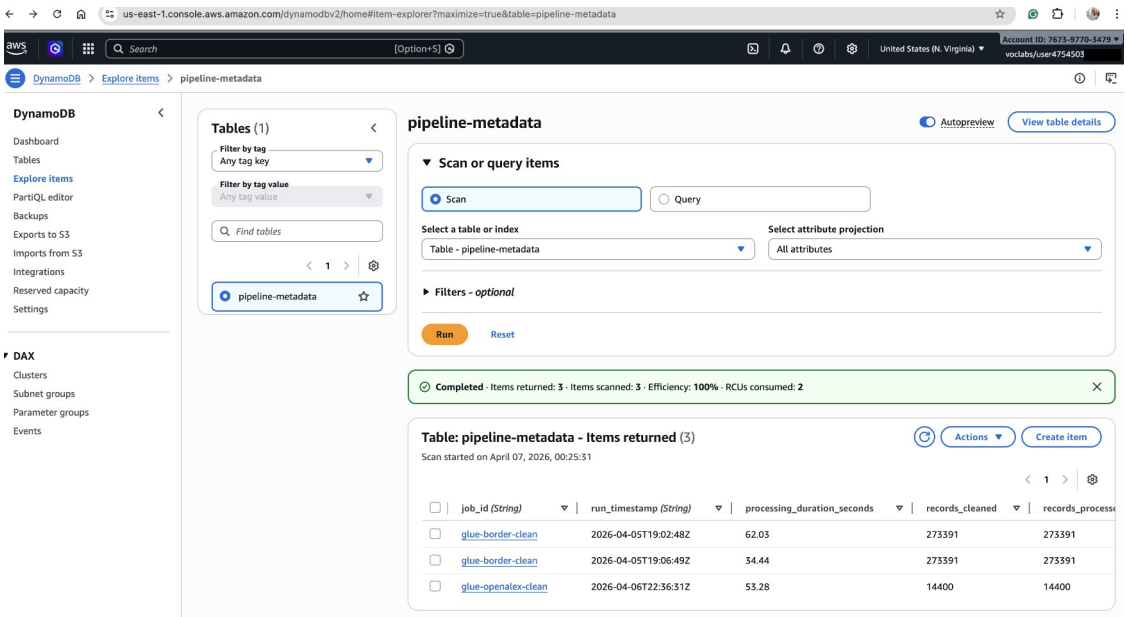


Figure 4: DynamoDB pipeline-metadata table with example log entries from successful runs

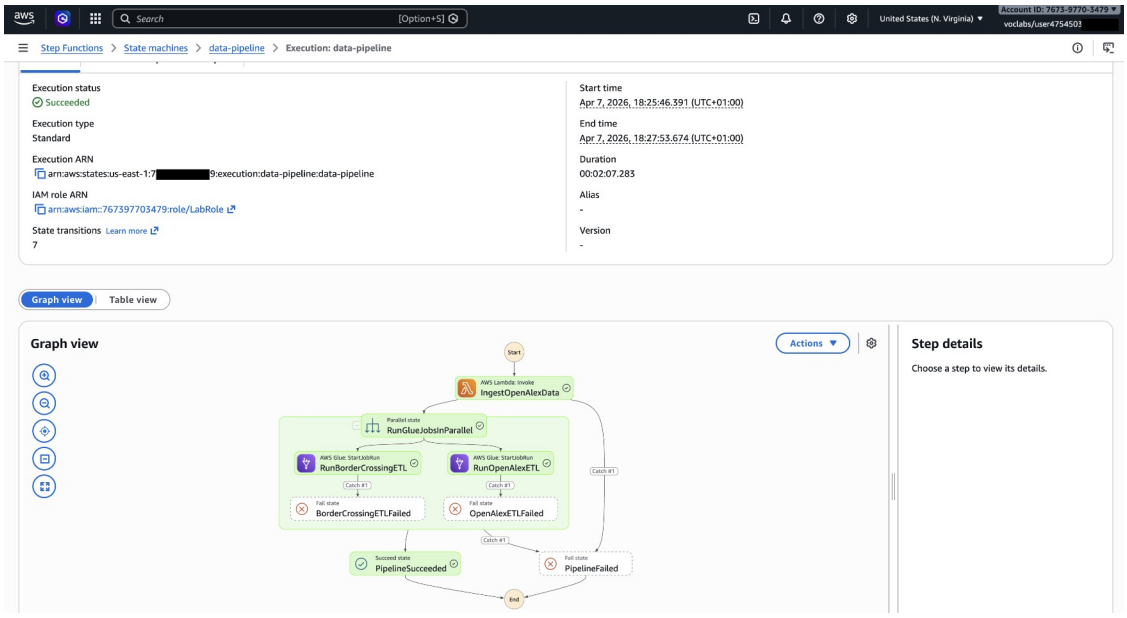


Figure 5: Successful Step Functions execution graph with all states green

---

Task 2 - Data Warehouse Development

1. Architecture overview

The data warehouse for Task 2 sits on top of the cleaned Parquet files produced in Task 1. The flow is short: the cleaned data lives in `s3://ae1-data-lake/cleaned/`, two Glue Crawlers scan those folders and register the schemas in the Glue Data Catalog under a database called `data_warehouse`, and Athena then queries those tables with standard SQL. There is no separate database server, no data load step, and no compute provisioning - Athena reads the Parquet files directly from S3 each time a query is run.

This follows the **schema-on-read** approach. The schema isn't baked into the files; it lives in the Catalog and gets applied when you actually query the data. If I want to change column types or add new columns, I just rerun the crawler, no data migration needed.

2. Cataloguing with Glue Crawlers

I created two crawlers (`border-crossing-crawler` and `openalex-crawler`), each pointing at the corresponding subfolder under `cleaned/` (**Figure 6, Figure 7**). Both run on demand using the `LabRole` IAM role and write into the `data_warehouse` database.

After running them, both `border_crossing` and `openalex` tables appeared in the Catalog with classification `parquet` (**Figure 8**). The crawler inferred all column types automatically from the Parquet metadata - `string` for text columns like `port name` and `state`, `int` for `value`, `date` for `date`, and `double` for `latitude` / `longitude` (**Figure 9**). One quirk worth noting: because the source CSV had headers like "Port Name" with a space, the crawler kept the

spaces in the column names, which means I have to quote them in SQL (`"port name"` instead of `port_name`). I left it that way to keep the catalog faithful to the source.

The `openalex` table was inferred similarly, with eight columns covering `work_id`, `title`, `publication_date`, `cited_by_count`, open access flags, and the comma-joined author/institution strings produced by the Glue ETL job (**Figure 10**).

3. Performance optimisation

Two design choices from Task 1 pay off heavily in Task 2:

Parquet columnar format. Athena only reads the columns referenced in a `SELECT` clause. When I queried `SELECT YEAR(date), SUM(value) FROM border_crossing`, Athena scanned just **1.01 MB** of data despite the table having over 273,000 rows across 9 columns (**Figure 11**). A `SELECT *` over the same table would scan many times more, because it would have to pull every column. This is the single most important reason to use Parquet over CSV in a lakehouse.

Snappy compression. Each Parquet file is Snappy-compressed on disk, so the bytes Athena has to read are smaller again. Snappy is not the best at compression ratio but it's fast to decompress, which matters more when Athena is scanning files on every query. Good enough compression with minimal overhead.

Partitioning would be the next step for a larger dataset. With 273k rows the warehouse is already small enough that partitioning would not change query costs much, but if I were ingesting millions of rows monthly I would partition `border_crossing` by year and `openalex` by `publication_date` year. Both fields are good partition candidates because most of my analytical queries filter or group by them.

4. Athena queries and analysis

I wrote five queries covering the analytical patterns the assignment asks for: trends over time, ranking, multi-dimensional aggregation, cross-format reading, and schema introspection. All five are saved in `task2-warehouse/athena-queries/`.

Query 1 - Trend over time (`01_yearly_border_crossings.sql`): aggregates total crossings by year. The result shows a steady drop from a peak around 1999–2000 (~527 million crossings) down through the early 2000s (**Figure 12**). Run time was ~523 ms, scanning ~1 MB.

Query 2 - Top 10 busiest ports (`02_top_busiest_ports.sql`): groups by port and state, sums the volume, ranks the result. San Ysidro (California) tops the list with ~1.45 billion total crossings, followed by El Paso, Laredo, and Hidalgo in Texas (**Figure 13**). This is where Parquet really helps, since Athena only had to read three columns out of nine, so the scan was tiny.

Query 3 - Top 20 cited OpenAlex papers (`03_top_cited_openalex_papers.sql`): orders the papers by `cited_by_count` (**Figure 14**). I noticed early on that the results contained duplicates - the same paper appearing several times. The cause is in the Lambda: it always starts pagination from cursor `*` so each scheduled run re-fetches the same first batch of recent works. Rather than rebuild the Lambda I added `DISTINCT` to the query, which is a reasonable schema-

on-read fix and shows the flexibility of querying raw lake data - I can patch logic at the query layer without touching storage.

Query 4 - Multi-dimensional aggregation (`04_yearly_crossings_by_border.sql`): splits totals by year *and* border (US-Canada vs US-Mexico), and additionally counts distinct ports and average crossings per record (**Figure 15**). This was the most interesting result: the US-Mexico border consistently sees roughly twice the volume of the US-Canada border across the time series.

Query 5 - Schema-on-read demonstration (`05_schema_on_read_demo.sql`): runs `DESCRIBE border_crossing` to print the schema (**Figure 16**) and `SHOW CREATE TABLE` to print the full external table definition including the S3 location, SerDe, and TBLPROPERTIES (**Figure 17**). The `SHOW CREATE TABLE` output is good evidence of schema-on-read: the table is declared as `EXTERNAL` and points at `s3://ae1-data-lake/cleaned/border_crossing/` - Athena does not own the data, it just reads it through the catalog definition.

5. Monitoring with CloudWatch

CloudWatch collects logs and metrics from every service in the pipeline without any manual setup. I used it to verify job execution history and query performance, which is what the spec asks for.

On the logging side, CloudWatch automatically created log groups for each service: `/aws/lambda/openalex-ingestion` for the Lambda function, `/aws-glue/jobs/output` for the Glue ETL output, `/aws-glue/jobs/error` for errors, and `/aws/kinesisfirehose/firehose-stream` for Firehose delivery (**Figure 20**). Drilling into the Lambda logs shows the full execution trace - the API date range, pagination progress ("Got 50 works, total: 200"), the final success response, and the REPORT line with duration (9.8s) and memory usage (78 of 128 MB) (**Figure 21**). The Glue logs are similarly detailed: raw row count, the full before/after schema, cleaned row count, output path confirmation, and DynamoDB logging duration (**Figure 22**).

For query performance, the Athena Recent Queries tab acts as a built-in monitoring dashboard - every query is listed with its status, run time, and data scanned (**Figure 23**). My queries consistently ran in under 1 second and scanned under 2 MB each. That is tiny, and it's because of Parquet and Snappy compression. Parquet is columnar, so Athena only reads the columns referenced in the SELECT clause rather than scanning entire rows. A query selecting 3 of 9 columns reads roughly a third of the data.

CloudWatch also tracks 282 numerical Glue job metrics automatically, including JVM heap usage, S3 bytes read/written, CPU system load, filesystem operations (**Figure 24, Figure 25**). These metrics exist without any configuration; they are published by the Glue service itself. The high-resolution data points may expire after ~15 days, but the log data persists indefinitely under the default retention policy.

Figures (Task 2)

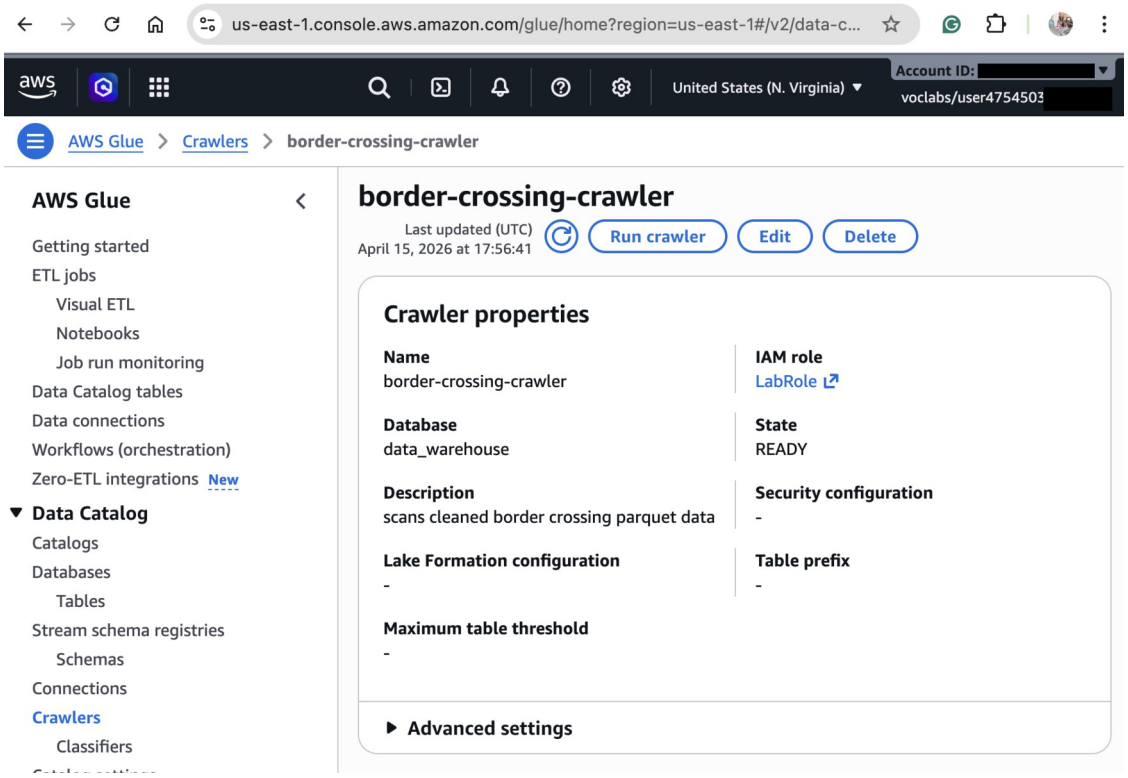


Figure 6: border-crossing-crawler configuration

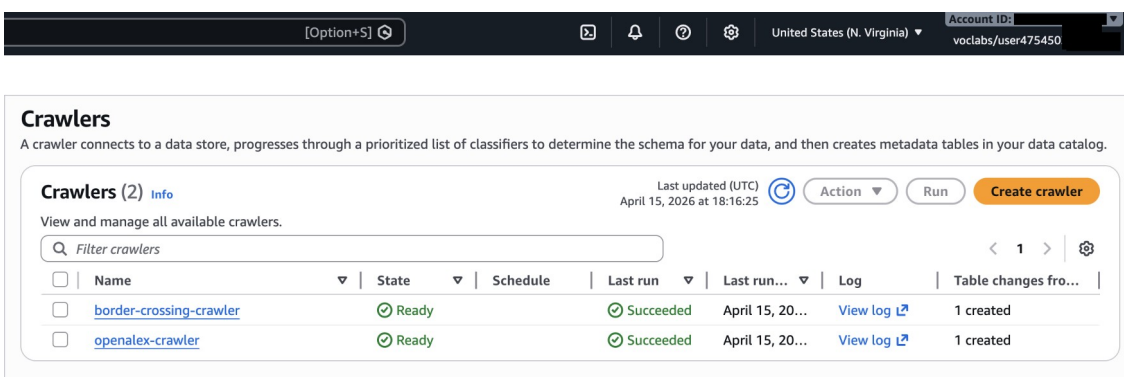


Figure 7: Both crawlers with status Succeeded

border_crossing

Table overview | Data quality - *new*

Table details

Name
border_crossing

Database
data_warehouse

Description
-

Last updated
April 15, 2026 at 18:01:57

Classification
Parquet

Location
s3://ae1-data-lake-[redacted]cleaned/border_crossing/

Connection
-

▶ **Advanced properties**

Schema | Partitions | Indexes | Column statistics - *new*

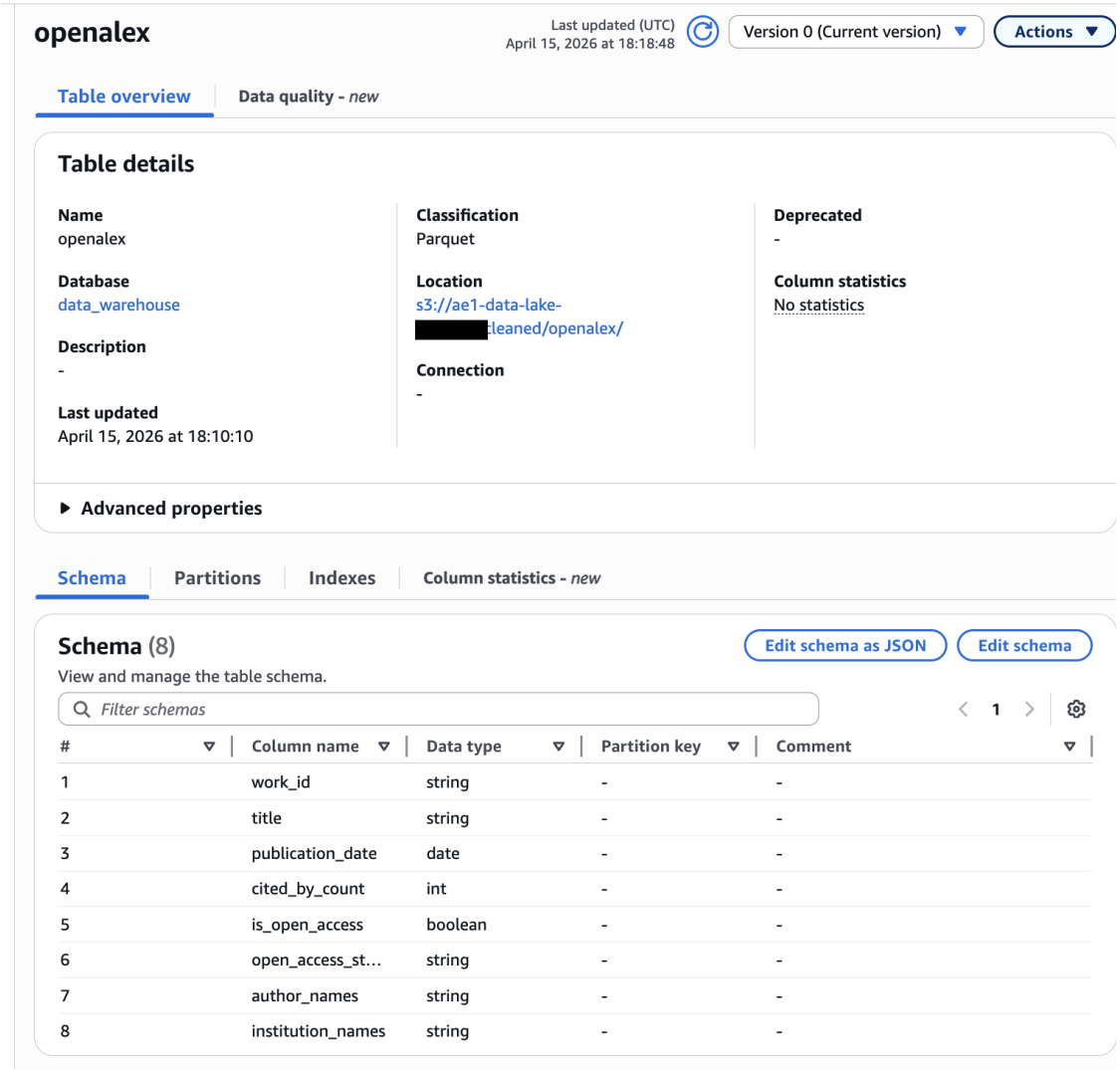
Schema (9)

View and manage the table schema.

Filter schemas

#	Column ...	Data type	Partition...	Comment
1	port name	string	-	-
2	state	string	-	-
3	port code	int	-	-
4	border	string	-	-
5	date	date	-	-
6	measure	string	-	-
7	value	int	-	-
8	latitude	double	-	-
9	longitude	double	-	-

Figure 9: border_crossing table schema



openalex

Last updated (UTC)
April 15, 2026 at 18:18:48

Version 0 (Current version) | Actions

Table overview | Data quality - *new*

Table details

Name
openalex

Database
data_warehouse

Description
-

Last updated
April 15, 2026 at 18:10:10

Classification
Parquet

Location
s3://ae1-data-lake-[redacted]leaned/openalex/

Connection
-

Deprecated
-

Column statistics
No statistics

▶ **Advanced properties**

Schema | Partitions | Indexes | Column statistics - *new*

Schema (8)

View and manage the table schema.

Edit schema as JSON | Edit schema

Filter schemas

1

#	Column name	Data type	Partition key	Comment
1	work_id	string	-	-
2	title	string	-	-
3	publication_date	date	-	-
4	cited_by_count	int	-	-
5	is_open_access	boolean	-	-
6	open_access_st...	string	-	-
7	author_names	string	-	-
8	institution_names	string	-	-

Figure 10: openalex table schema

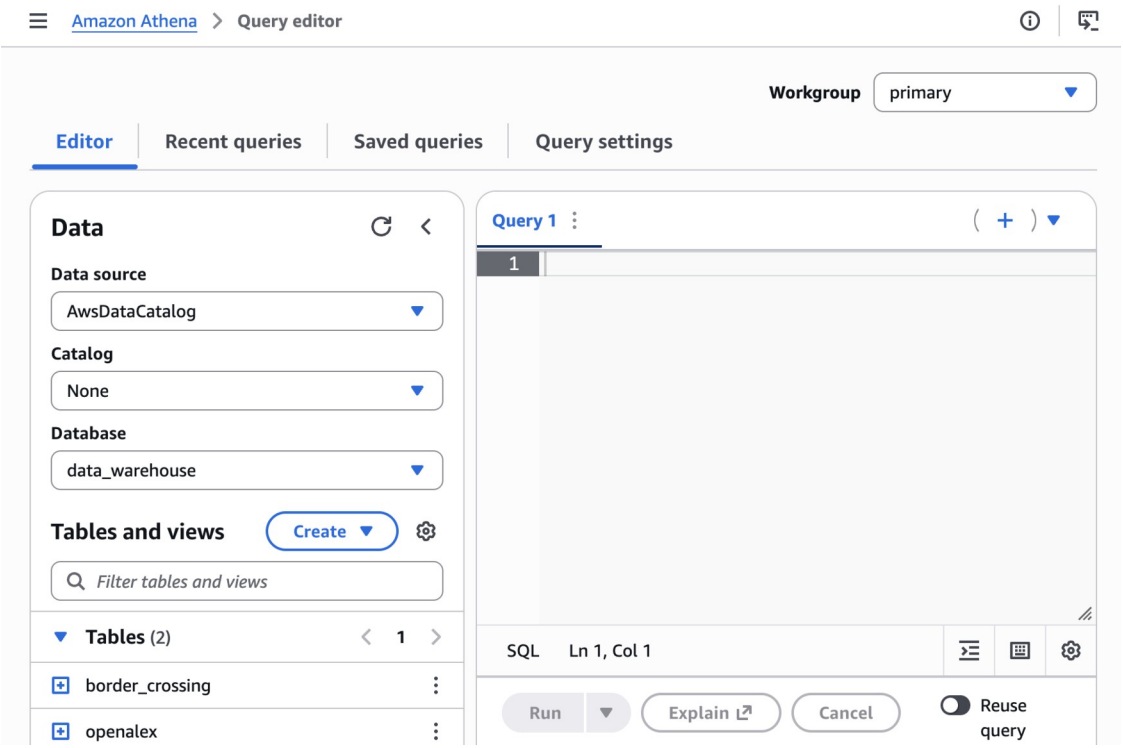


Figure 11: Athena query editor with data_warehouse

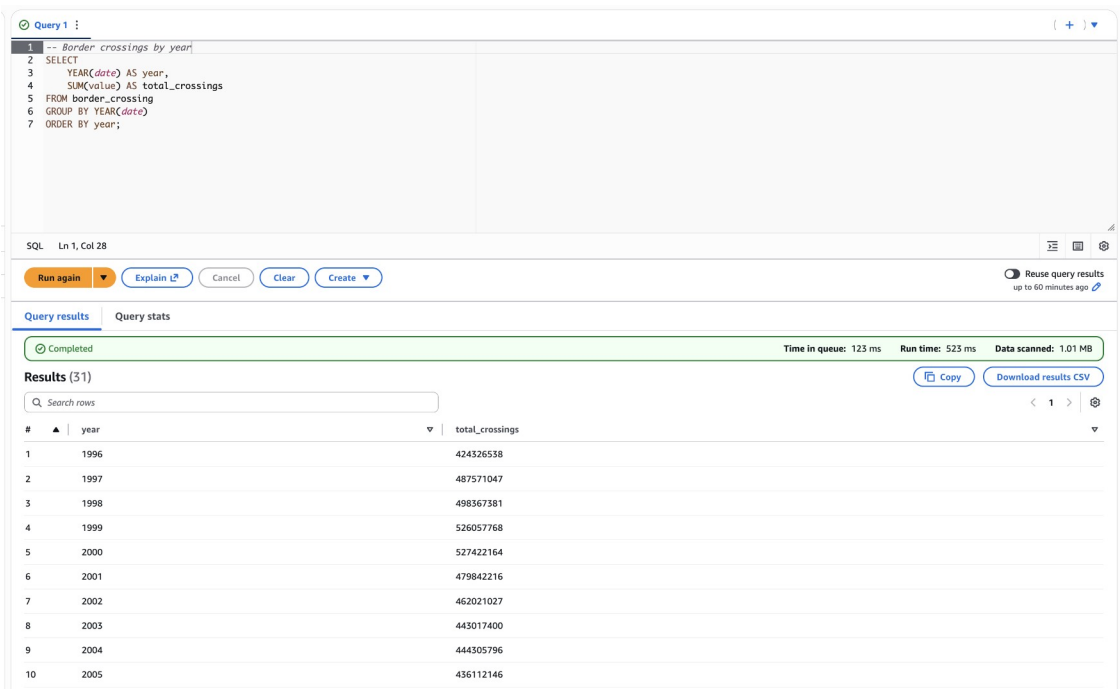


Figure 12: Yearly border crossings trend

Query 1 | Query 2

```sql
-- Top 10 busiest border crossing ports
SELECT
    "port name",
    state,
    SUM(value) AS total
FROM border_crossing
GROUP BY "port name", state
ORDER BY total DESC
LIMIT 10;
```

SQL Ln 7, Col 21

Run again | Explain | Cancel | Clear | Create | Reuse query results up to 60 minutes ago

Query results | Query stats

Completed **Time in queue:** 108 ms **Run time:** 627 ms **Data scanned:** 1.12 MB

Results (10) Copy | Download results CSV

Search rows 1

#	port name	state	total
1	San Ysidro	California	1449798032
2	El Paso	Texas	1308744295
3	Laredo	Texas	741498521
4	Hidalgo	Texas	660325848
5	Calexico	California	630228448
6	Brownsville	Texas	612299728
7	Buffalo Niagara Falls	New York	592241423
8	Otay Mesa	California	562269475
9	Detroit	Michigan	506840195
10	Nogales	Arizona	481449582

Figure 13: Top 10 busiest border crossing ports

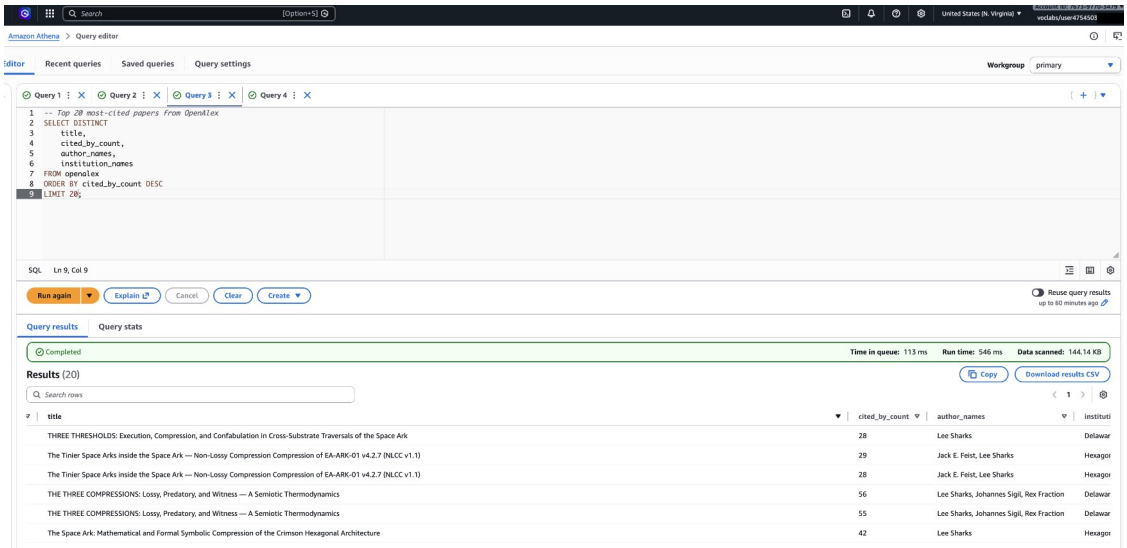


Figure 14: Top 20 most-cited OpenAlex papers

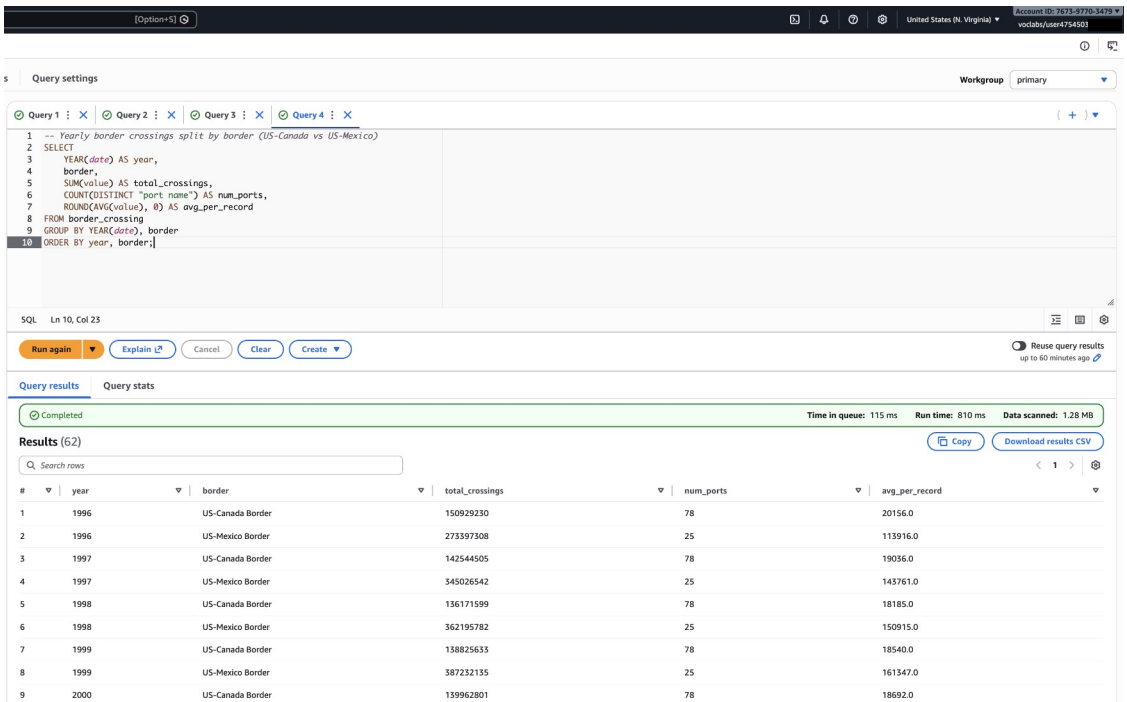


Figure 15: Yearly crossings split by border

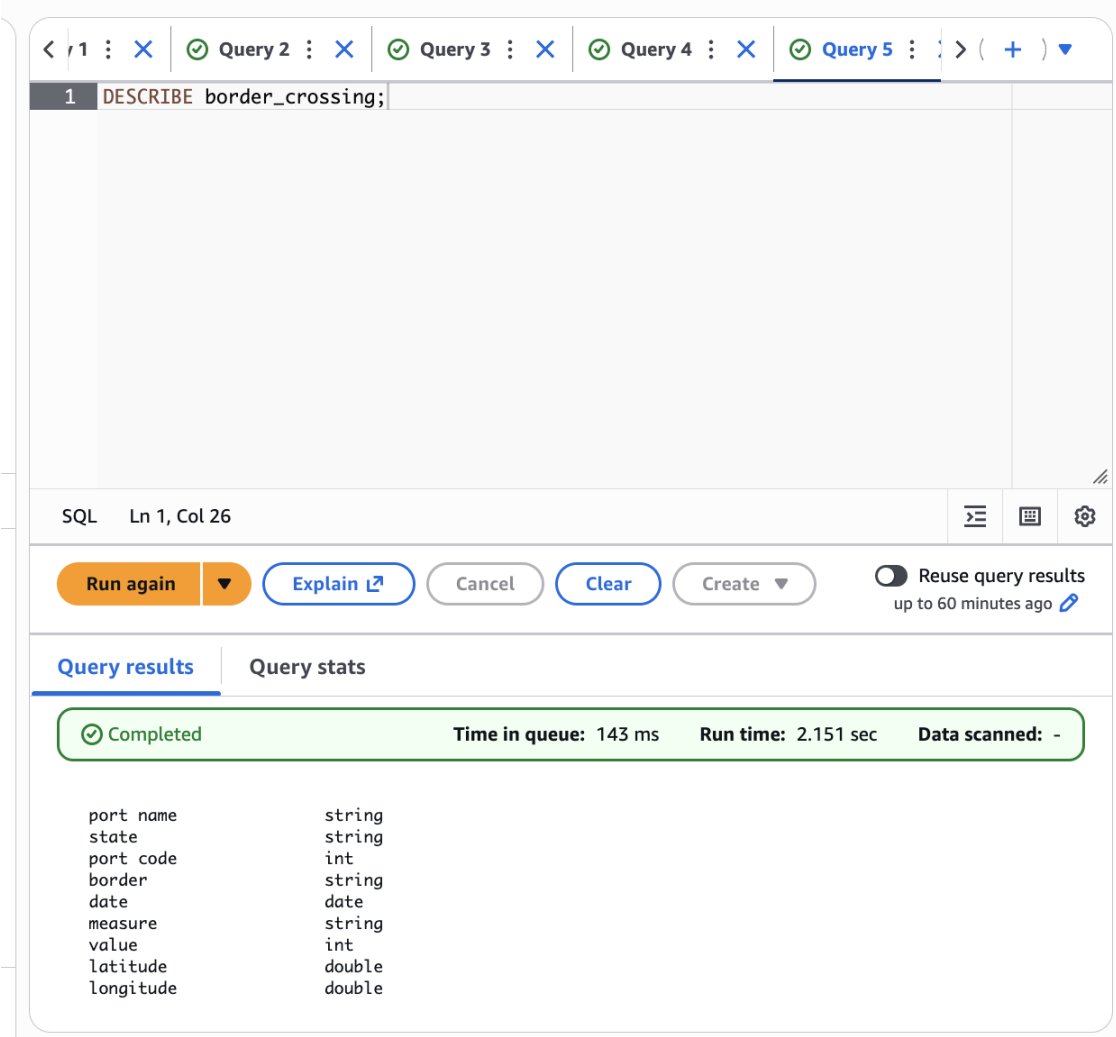


Figure 16: DESCRIBE border_crossing showing schema-on-read

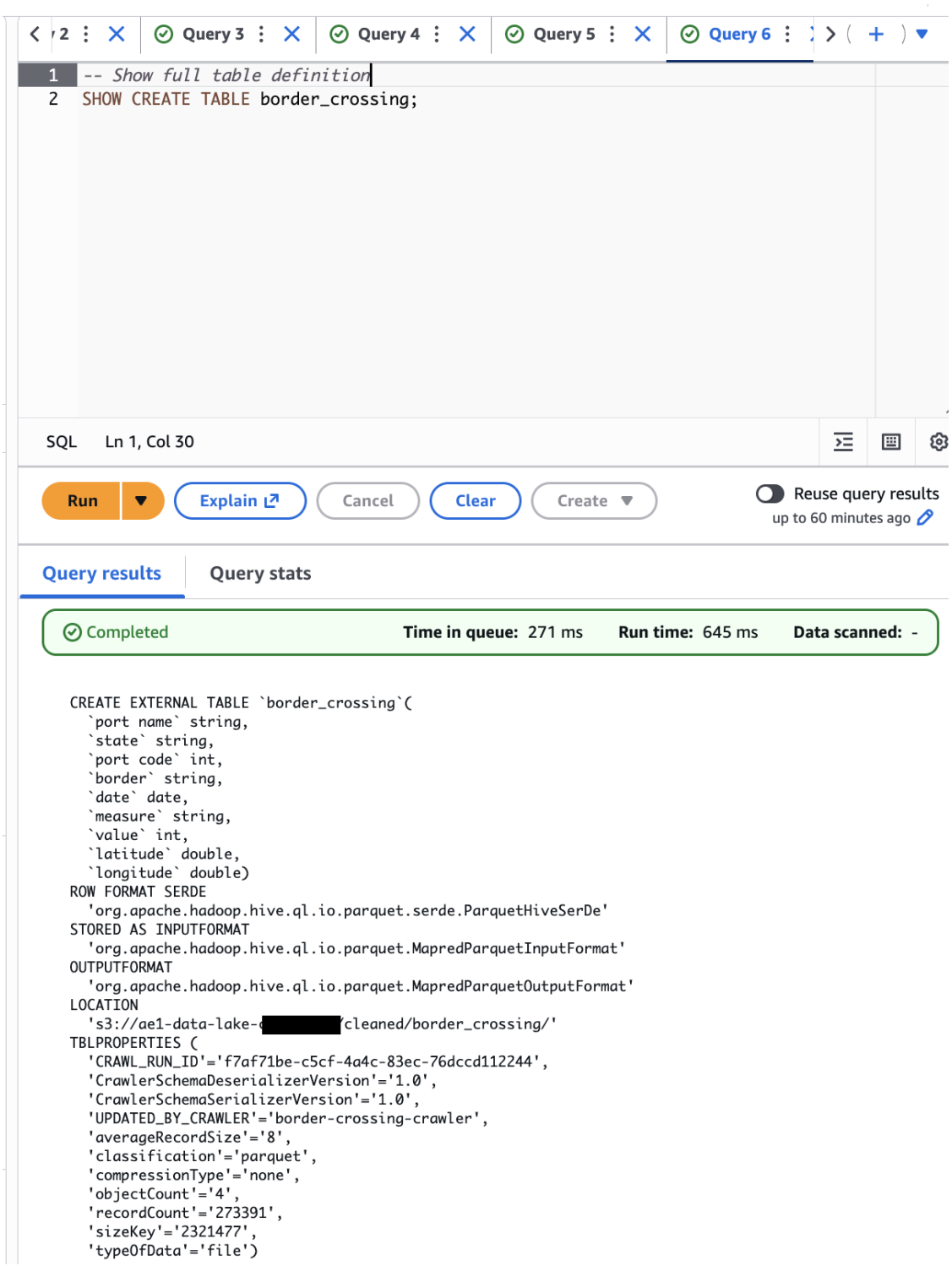
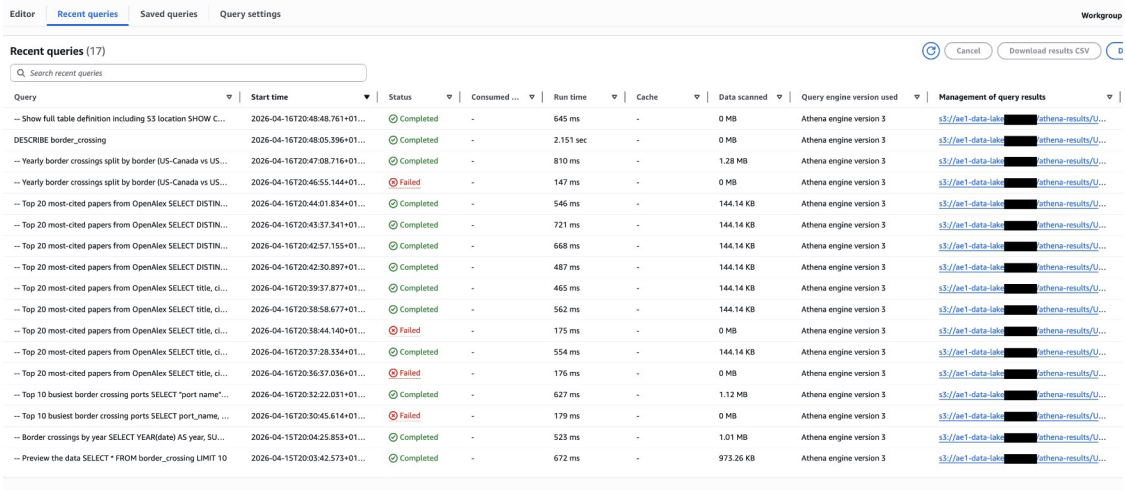


Figure 17: SHOW CREATE TABLE output



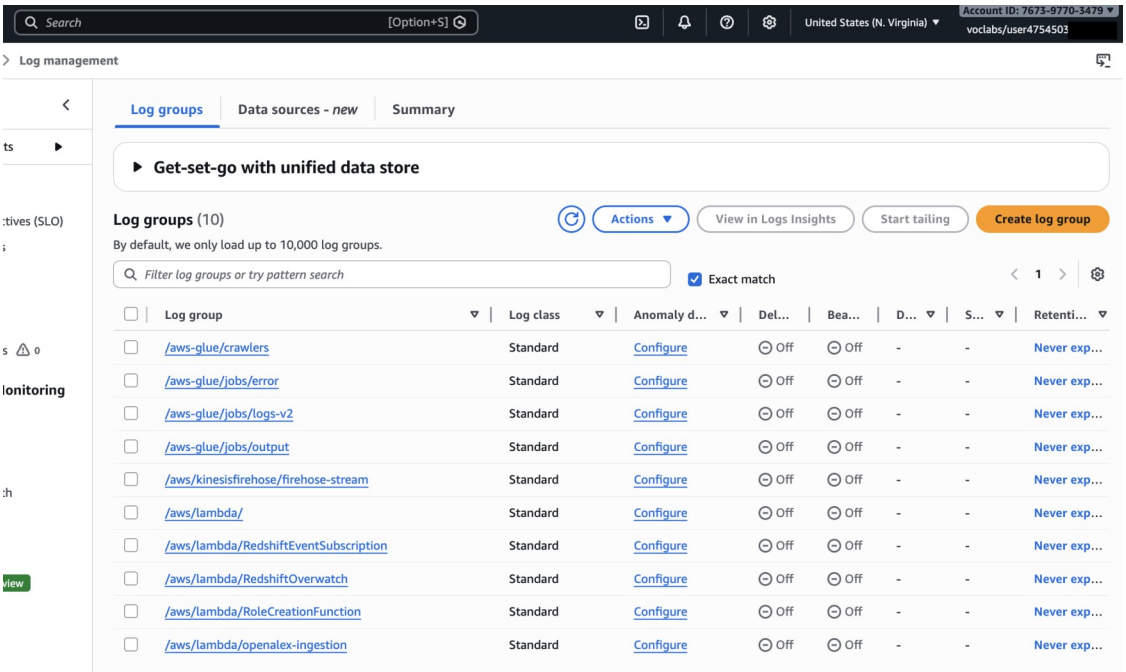
Editor | Recent queries | Saved queries | Query settings | Workgroup

Recent queries (17)

Search recent queries

Query	Start time	Status	Consumed ...	Run time	Cache	Data scanned	Query engine version used	Management of query results
-- Show full table definition including S3 location SHOW C...	2026-04-16T20:48:48.761+01...	Completed	-	645 ms	-	0 MB	Athena engine version 3	s3://ae1-data-lake athena-results/U...
DESCRIBE border_crossing	2026-04-16T20:48:05.396+01...	Completed	-	2.151 sec	-	0 MB	Athena engine version 3	s3://ae1-data-lake athena-results/U...
-- Yearly border crossings split by border (US-Canada vs US...	2026-04-16T20:47:08.716+01...	Completed	-	810 ms	-	1.28 MB	Athena engine version 3	s3://ae1-data-lake athena-results/U...
-- Yearly border crossings split by border (US-Canada vs US...	2026-04-16T20:46:55.144+01...	Failed	-	147 ms	-	0 MB	Athena engine version 3	s3://ae1-data-lake athena-results/U...
-- Top 20 most-cited papers from OpenAlex SELECT DISTIN...	2026-04-16T20:44:01.834+01...	Completed	-	546 ms	-	144.14 KB	Athena engine version 3	s3://ae1-data-lake athena-results/U...
-- Top 20 most-cited papers from OpenAlex SELECT DISTIN...	2026-04-16T20:43:37.341+01...	Completed	-	721 ms	-	144.14 KB	Athena engine version 3	s3://ae1-data-lake athena-results/U...
-- Top 20 most-cited papers from OpenAlex SELECT DISTIN...	2026-04-16T20:42:57.155+01...	Completed	-	668 ms	-	144.14 KB	Athena engine version 3	s3://ae1-data-lake athena-results/U...
-- Top 20 most-cited papers from OpenAlex SELECT DISTIN...	2026-04-16T20:42:30.897+01...	Completed	-	487 ms	-	144.14 KB	Athena engine version 3	s3://ae1-data-lake athena-results/U...
-- Top 20 most-cited papers from OpenAlex SELECT title, ci...	2026-04-16T20:39:37.877+01...	Completed	-	465 ms	-	144.14 KB	Athena engine version 3	s3://ae1-data-lake athena-results/U...
-- Top 20 most-cited papers from OpenAlex SELECT title, ci...	2026-04-16T20:38:58.677+01...	Completed	-	562 ms	-	144.14 KB	Athena engine version 3	s3://ae1-data-lake athena-results/U...
-- Top 20 most-cited papers from OpenAlex SELECT title, ci...	2026-04-16T20:38:44.140+01...	Failed	-	175 ms	-	0 MB	Athena engine version 3	s3://ae1-data-lake athena-results/U...
-- Top 20 most-cited papers from OpenAlex SELECT title, ci...	2026-04-16T20:37:28.334+01...	Completed	-	554 ms	-	144.14 KB	Athena engine version 3	s3://ae1-data-lake athena-results/U...
-- Top 20 most-cited papers from OpenAlex SELECT title, ci...	2026-04-16T20:36:37.036+01...	Failed	-	176 ms	-	0 MB	Athena engine version 3	s3://ae1-data-lake athena-results/U...
-- Top 10 busiest border crossing ports SELECT "port name"...	2026-04-16T20:32:22.031+01...	Completed	-	627 ms	-	1.12 MB	Athena engine version 3	s3://ae1-data-lake athena-results/U...
-- Top 10 busiest border crossing ports SELECT port_name, ...	2026-04-16T20:30:45.614+01...	Failed	-	179 ms	-	0 MB	Athena engine version 3	s3://ae1-data-lake athena-results/U...
-- Border crossings by year SELECT YEAR(date) AS year, SU...	2026-04-15T20:04:25.853+01...	Completed	-	523 ms	-	1.01 MB	Athena engine version 3	s3://ae1-data-lake athena-results/U...
-- Preview the data SELECT * FROM border_crossing LIMIT 10	2026-04-15T20:03:42.573+01...	Completed	-	672 ms	-	973.26 KB	Athena engine version 3	s3://ae1-data-lake athena-results/U...

Figure 18: Athena Recent Queries



Search [Option+S] | United States (N. Virginia) | Account ID: 7673-9770-3479 voclabs/user4754503

Log management

Log groups | Data sources - *new* | Summary

Get-set-go with unified data store

Log groups (10)

By default, we only load up to 10,000 log groups.

Actions | View in Logs Insights | Start tailing | Create log group

Filter log groups or try pattern search | Exact match

Log group	Log class	Anomaly d...	Del...	Bea...	D...	S...	Retenti...
/aws-glue/crawlers	Standard	Configure	Off	Off	-	-	Never exp...
/aws-glue/jobs/error	Standard	Configure	Off	Off	-	-	Never exp...
/aws-glue/jobs/logs-v2	Standard	Configure	Off	Off	-	-	Never exp...
/aws-glue/jobs/output	Standard	Configure	Off	Off	-	-	Never exp...
/aws/kinesisfirehose/firehose-stream	Standard	Configure	Off	Off	-	-	Never exp...
/aws/lambda/	Standard	Configure	Off	Off	-	-	Never exp...
/aws/lambda/RedshiftEventSubscription	Standard	Configure	Off	Off	-	-	Never exp...
/aws/lambda/RedshiftOverwatch	Standard	Configure	Off	Off	-	-	Never exp...
/aws/lambda/RoleCreationFunction	Standard	Configure	Off	Off	-	-	Never exp...
/aws/lambda/openalex-ingestion	Standard	Configure	Off	Off	-	-	Never exp...

Figure 19: CloudWatch Log groups

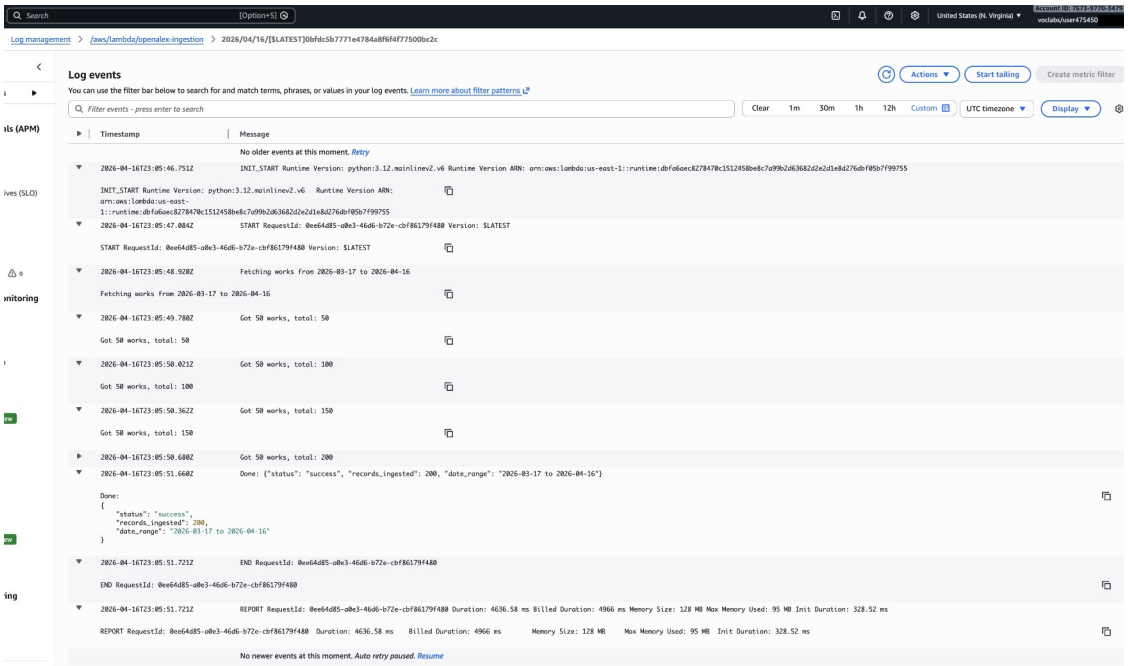
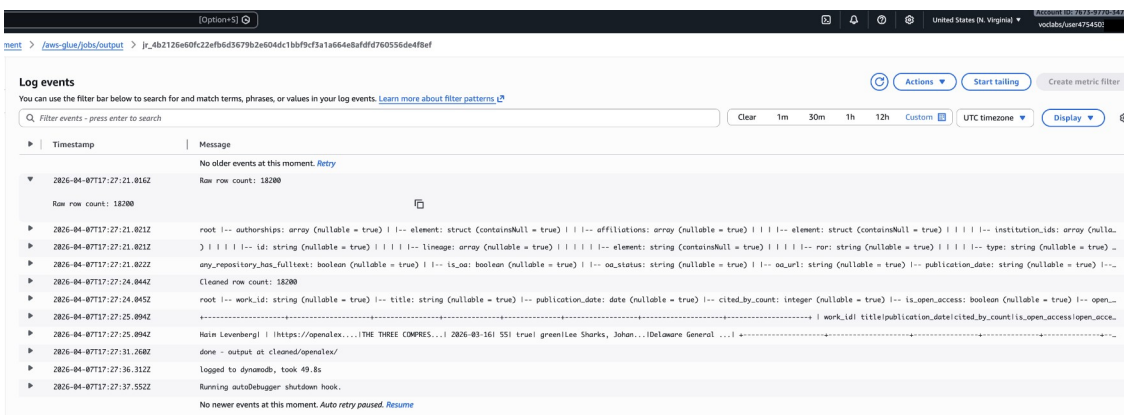


Figure 21: Lambda execution logs


/aws-glue/jobs/output > jr_4b2126e60fc22efb6d3679b2e604dc1bbf9cf3a1a664e8afdfd760556de4f8ef
Log events
You can use the filter bar below to search for and match terms, phrases, or values in your log events. Learn more about filter patterns
Actions
Start tailing
Create metric filter
Timestamp
Message
No older events at this moment. Retry
2026-04-07T17:27:21.016Z Raw row count: 18200
Raw row count: 18200
2026-04-07T17:27:21.021Z root |-- authorships: array (nullable = true) | |-- element: struct (containsNull = true) | | |-- affiliations: array (nullable = true) | | | |-- element: struct (containsNull = true) | | | | |-- institution_ids: array (nulla…
2026-04-07T17:27:21.021Z) | | | | |-- id: string (nullable = true) | | | | |-- lineage: array (nullable = true) | | | | | |-- element: string (containsNull = true) | | | | |-- ror: string (nullable = true) | | | | |-- type: string (nullable = true) …
2026-04-07T17:27:21.022Z any_repository_has_fulltext: boolean (nullable = true) | |-- is_oa: boolean (nullable = true) | |-- oa_status: string (nullable = true) | |-- oa_url: string (nullable = true) |-- publication_date: string (nullable = true) |--…
2026-04-07T17:27:24.044Z Cleaned row count: 18200
2026-04-07T17:27:24.045Z root |-- work_id: string (nullable = true) |-- title: string (nullable = true) |-- publication_date: date (nullable = true) |-- cited_by_count: integer (nullable = true) |-- is_open_access: boolean (nullable = true) |-- open_…
2026-04-07T17:27:25.094Z +----------+----------+----------+

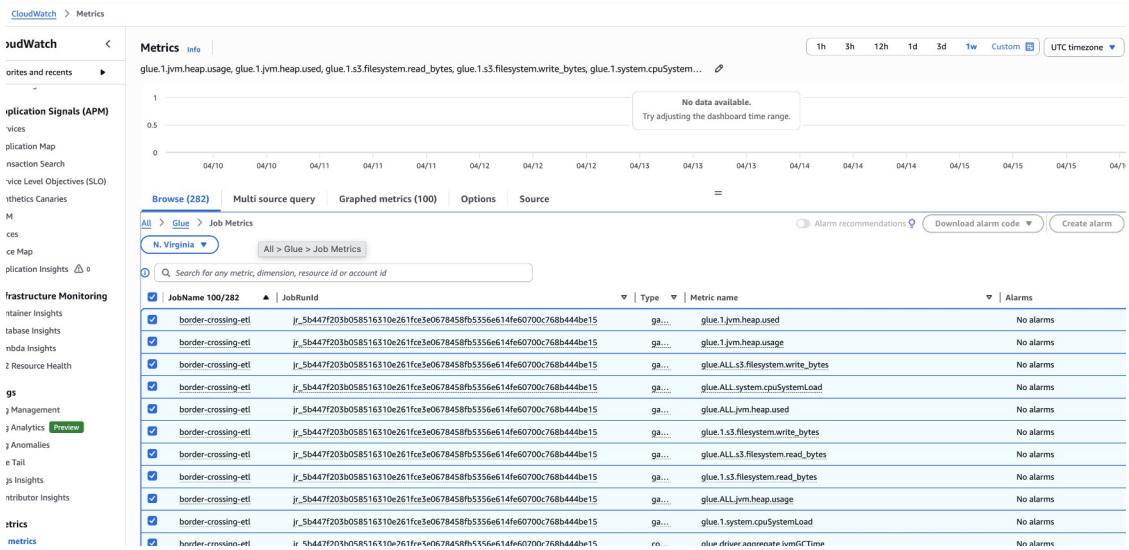


Figure 23: CloudWatch Metrics page

Task 3 - Workflow Automation & Visualisation

1. Extending the Step Functions workflow

The Task 1 pipeline stopped after the Glue ETL jobs wrote cleaned Parquet to S3, which meant I still had to manually run the crawlers and then run the Athena query myself any time I wanted fresh results. For Task 3 I extended the same state machine so the pipeline goes from raw ingestion all the way through to a queryable CSV without any manual steps.

Three new states sit after the parallel ETL jobs (**Figure 26**). **RunCrawlersInParallel** triggers both Glue Crawlers at the same time so the Catalog picks up any schema changes from the freshly written Parquet. **WaitForCrawlers** pauses 60 seconds, since crawlers don't support the `.sync` integration the way Glue jobs do. **RunAthenaQuery** then runs an aggregation query and writes the result as a CSV to `s3://ae1-data-lake/athena-results/`. The full definition is in `task3-visualisation/step-functions/workflow-definition.json`.

Each new state has the same `Catch` block pattern from Task 1, routing to `PipelineFailed` if anything errors. The `.sync` integration on the Athena step is what kept this clean: without it I would have needed a Wait + GetQueryExecution + Choice polling loop. The 60-second crawler wait is a workaround; for a much bigger dataset I'd build an actual status polling loop.

The full extended pipeline ran end-to-end in about 3 minutes 52 seconds with every state green (**Figure 27**), and the CSV showed up in S3 right after (**Figure 28**).

2. Athena query design

The query the workflow runs is shaped for downstream use:

```sql
SELECT state, border, measure, SUM(value) AS total_crossings
FROM data_warehouse.border_crossing
GROUP BY state, border, measure
ORDER BY total_crossings DESC
```

It returns 114 rows, one per state/border/measure combination. I picked that granularity on purpose: dumping all 273k raw rows into a visualisation tool would be slow and most charts aggregate anyway, so pre-aggregating in Athena keeps the output small but still flexible enough to slice by state, border, or measure. The query ran in under 600 ms scanning ~1 MB, which lines up with the Parquet performance from Task 2.

3. Monitoring

The extended pipeline reuses the monitoring setup from Task 2 with no extra config. Step Functions logs every state transition with timing and payloads, the new crawler runs land in `/aws-glue/crawlers`, and the Athena query shows up in Recent Queries with its scan size. The DynamoDB metadata table from Task 1 still captures every Glue job run, so combined with Step Functions history and CloudWatch logs I have full traceability from the API call all the way to the final CSV.

Figures (Task 3)

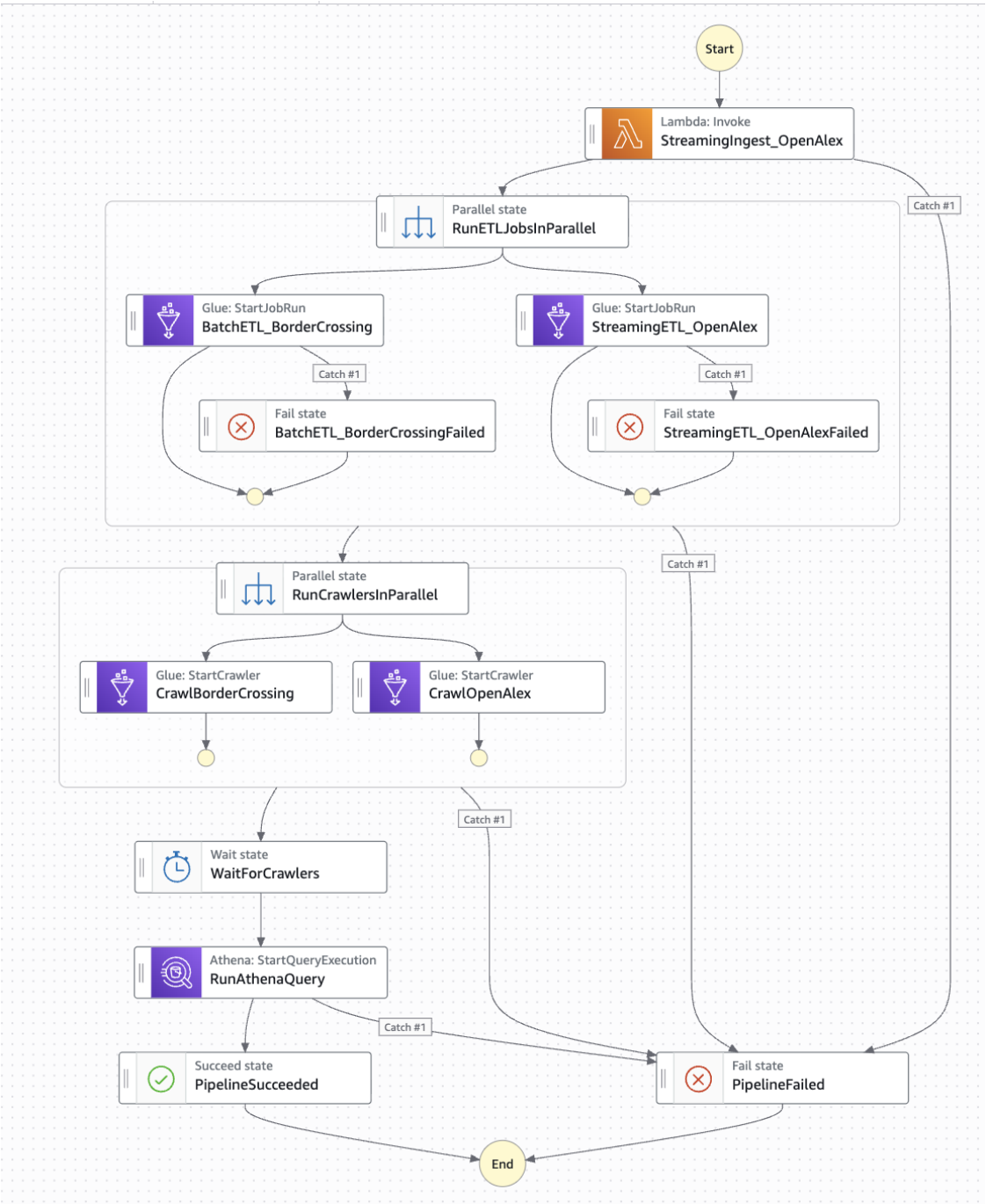


Figure 26: Extended Step Functions workflow with crawlers and Athena

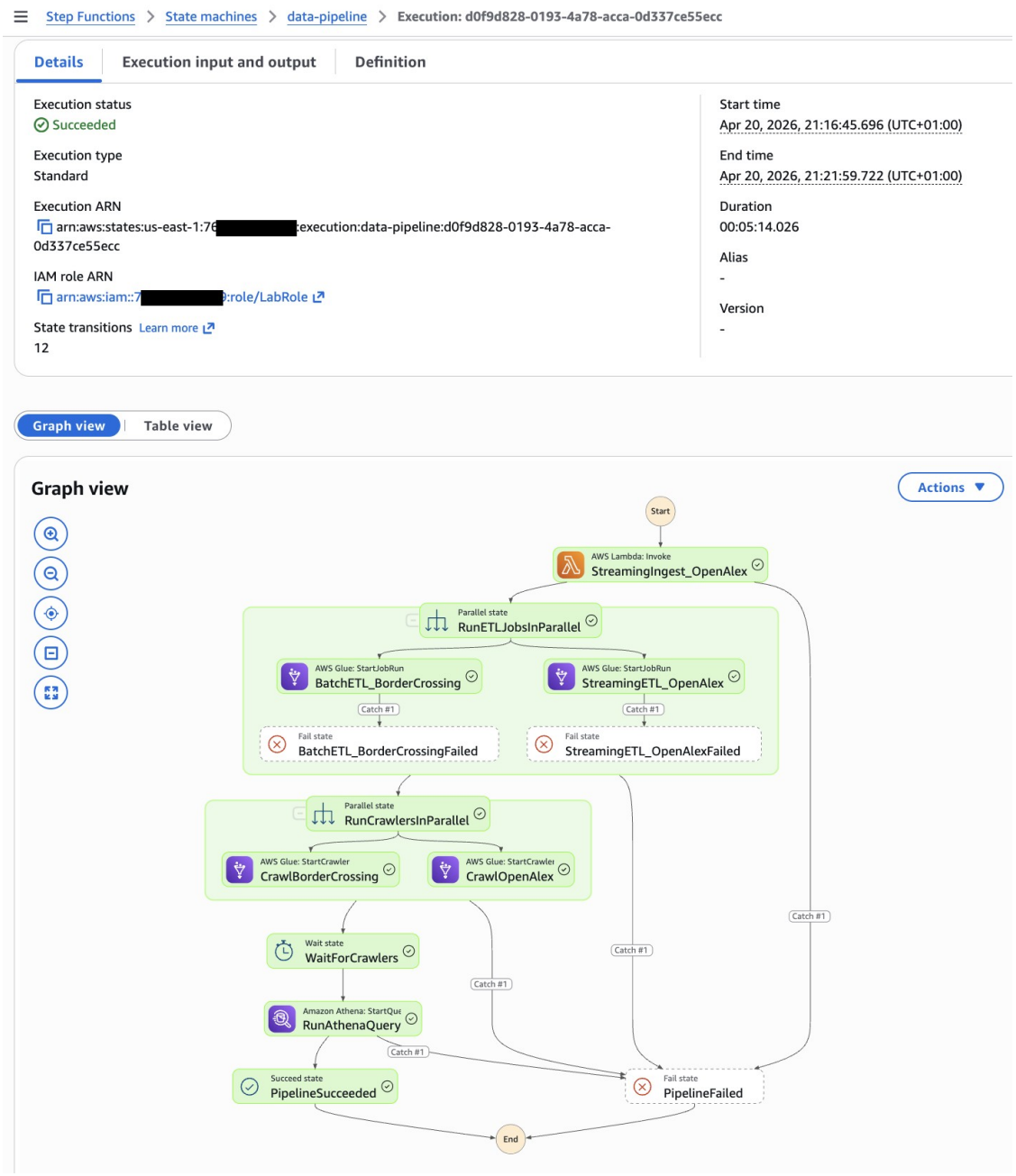


Figure 27: Successful execution of extended pipeline with duration and details

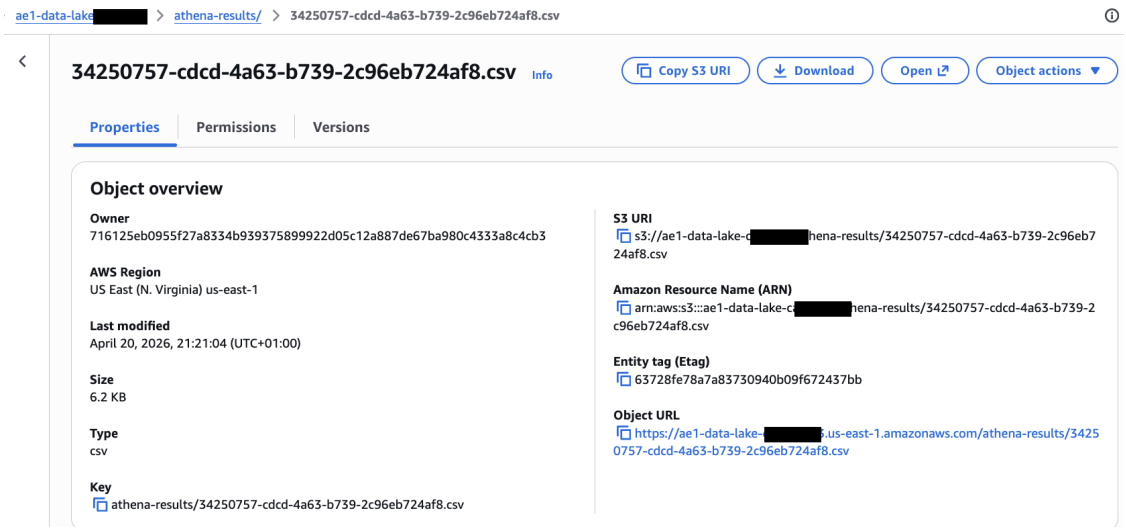
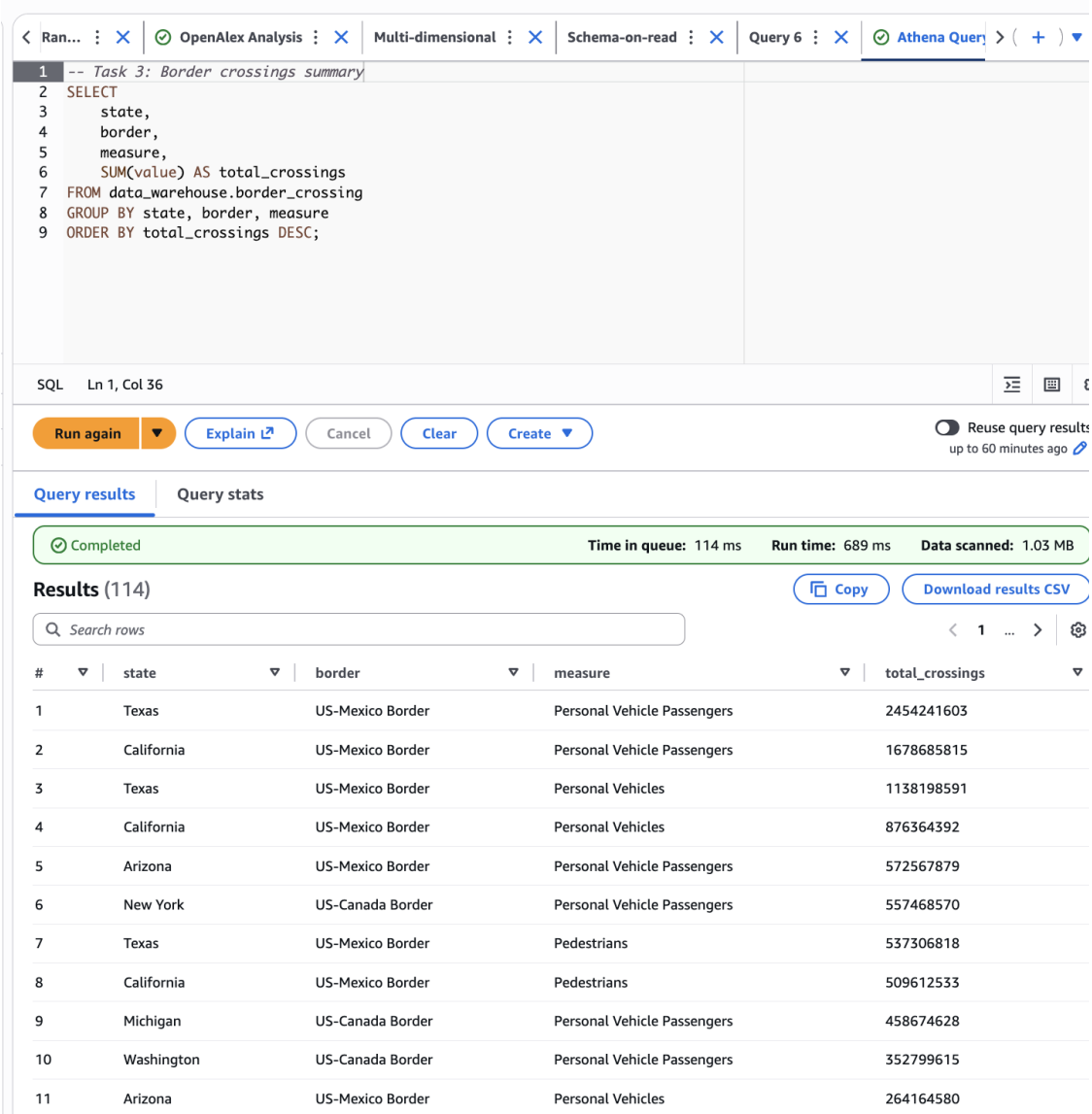


Figure 28: S3 showing Athena result CSV in athena-results/



```sql
-- Task 3: Border crossings summary
SELECT
    state,
    border,
    measure,
    SUM(value) AS total_crossings
FROM data_warehouse.border_crossing
GROUP BY state, border, measure
ORDER BY total_crossings DESC;
```

Ran... | OpenAlex Analysis | Multi-dimensional | Schema-on-read | Query 6 | Athena Query

SQL Ln 1, Col 36

Run again | Explain | Cancel | Clear | Create

Reuse query results up to 60 minutes ago

Query results | Query stats

Completed — Time in queue: 114 ms — Run time: 689 ms — Data scanned: 1.03 MB

Results (114) — Copy | Download results CSV

Search rows — 1 ...

#	state	border	measure	total_crossings
1	Texas	US-Mexico Border	Personal Vehicle Passengers	2454241603
2	California	US-Mexico Border	Personal Vehicle Passengers	1678685815
3	Texas	US-Mexico Border	Personal Vehicles	1138198591
4	California	US-Mexico Border	Personal Vehicles	876364392
5	Arizona	US-Mexico Border	Personal Vehicle Passengers	572567879
6	New York	US-Canada Border	Personal Vehicle Passengers	557468570
7	Texas	US-Mexico Border	Pedestrians	537306818
8	California	US-Mexico Border	Pedestrians	509612533
9	Michigan	US-Canada Border	Personal Vehicle Passengers	458674628
10	Washington	US-Canada Border	Personal Vehicle Passengers	352799615
11	Arizona	US-Mexico Border	Personal Vehicles	264164580

Figure 29: Athena query results showing 114 rows with state, border, measure, total_crossings